

Readora- BookStore

INTRODUCTION

Readora is a MERN-based online bookstore that offers an easy and secure platform for browsing, purchasing, and managing books. It provides an intuitive user experience with powerful admin features for inventory and order management.

DESCRIPTION

Readora – MERN Bookstore is a user-friendly full-stack web application designed to simplify the process of discovering, purchasing, and managing books online. It provides a centralized platform where users can securely register and log in, browse books by category, search for their favorite titles, view detailed book information, add books to their shopping cart, and place orders conveniently. The platform also includes an administrative dashboard that enables administrators to efficiently manage the bookstore inventory by adding, updating, and deleting books while monitoring sales statistics and revenue analytics.

The application is developed using the MERN Stack (MongoDB, Express.js, React.js, and Node.js) with Firebase Authentication for secure user authentication, Redux Toolkit for state management, Tailwind CSS for responsive user interface design, and Chart.js for dashboard analytics. The system follows the MVC (Model–View–Controller) architecture to ensure modularity, maintainability, scalability, and efficient communication between the frontend, backend, and database.

It helps users discover books quickly while enabling administrators to efficiently manage inventory and customer orders. By providing secure authentication, responsive design, efficient REST APIs, and interactive dashboards, the application ensures a seamless online bookstore experience for both customers and administrators.

Key Features

1. **User Registration & Authentication:** Users can create accounts and securely log in using Firebase Authentication.
2. **Book Browsing & Search:** Users can browse books by category, search for books using keywords, and view detailed information about each book.
3. **Shopping Cart:** Users can add books to their shopping cart, remove books, clear the cart, and manage selected items before checkout.
4. **Order Management:** Users can place book orders and view their order history.
5. **Admin Dashboard:** Administrators can add, update, and delete books while monitoring revenue statistics, latest orders, and overall bookstore performance.
6. **Security & Data Management:** The application ensures secure authentication,

protected admin access, reliable data storage using MongoDB Atlas, and efficient communication through REST APIs.

SCENARIO-BASED CASE STUDY

Scenario: Sarah, a college student who enjoys reading books, wants to purchase a programming book from Readora – MERN Bookstore.

1. User Registration and Login:

- Sarah visits the Readora Bookstore website and clicks the Sign-Up button to create a new account.
- She enters her name, email address, and password to complete registration.
- After successful registration, Sarah logs into the application using Firebase Authentication.
- Once authenticated, she is redirected to the homepage where she can explore the available books.

2. Book Browsing & Search:

- Sarah browses different book categories such as Technology, Fiction, Business, and Self-Development.
- She searches for “Leading Digital” using the search bar.
- The application displays the matching book along with its cover image, price, author, category, and description.

3. Shopping Cart & Checkout:

- Sarah opens the book details page and clicks Add to Cart.
- The selected book is added to her shopping cart successfully.
- She reviews the cart, verifies the selected items, and proceeds to the checkout page.
- The order is successfully placed and stored in the database.

4. Order Management:

- Sarah visits the Orders page to view her previously placed orders.
- She can review the order details and confirm that her purchase has been recorded successfully.

5. Admin Management:

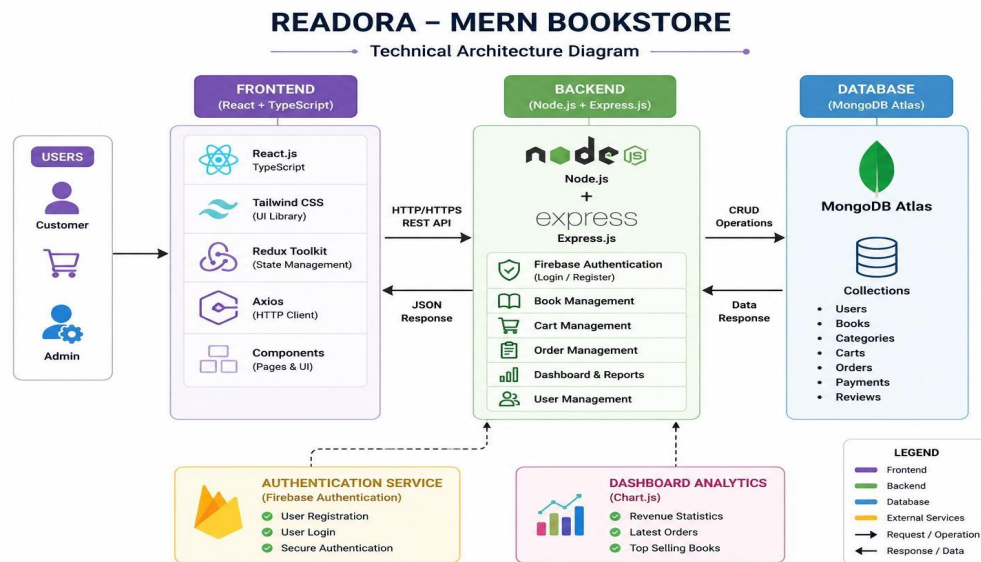
- The administrator logs into the Admin Dashboard.
- The admin adds new books, updates book information, edits prices, and removes unavailable books from the inventory.
- The administrator monitors revenue statistics, latest orders, and dashboard analytics to efficiently manage the bookstore

6. Security & System Maintenance

- Firebase Authentication securely manages user login and registration.

- MongoDB Atlas securely stores information related to books, users, and orders.
- REST APIs ensure smooth communication between the frontend and backend.
- The MVC architecture keeps the application modular, scalable, and easy to maintain while delivering a secure and reliable online bookstore experience.

TECHNICAL ARCHITECTURE

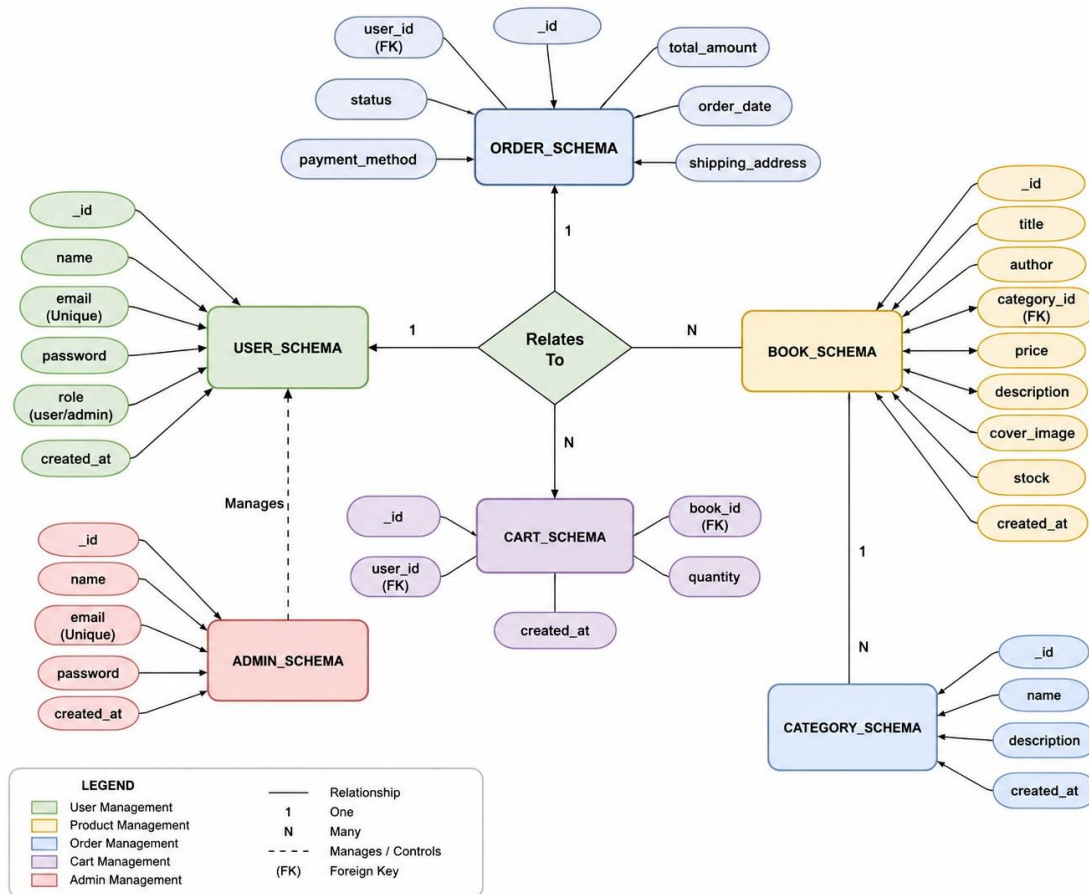


The technical architecture of Readora – MERN Bookstore follows a client-server architecture, where the frontend acts as the client, and the backend functions as the server. The frontend is developed using React.js with TypeScript, providing a responsive and interactive user interface for both customers and administrators. Redux Toolkit is used for efficient state management, while Tailwind CSS ensures a modern, responsive, and user-friendly design. The frontend communicates with the backend through RESTful APIs using Axios for seamless data exchange. On the backend, Node.js and Express.js are used to implement server-side logic, manage API requests, handle business operations, and process user interactions. The application follows the MVC (Model–View–Controller) architecture, making the code modular, maintainable, and scalable.

For secure user authentication, the application integrates Firebase Authentication, allowing users to register and log in securely. Administrative operations such as book management and dashboard access are protected through authentication and authorization mechanisms. For data storage and retrieval, the backend uses MongoDB Atlas, which efficiently stores user information, book details, shopping cart data, orders, and other application records. MongoDB provides reliable, scalable, and high-performance database services, ensuring quick access to data during book browsing, cart management, order processing, and administrative operations. Together, the React.js frontend, Node.js & Express.js backend, Firebase Authentication, MongoDB Atlas, Redux Toolkit, and RESTful APIs form a robust and scalable architecture for the Readora Bookstore application. This architecture enables secure authentication, efficient data

management, fast communication between the frontend and backend, and a seamless online bookstore experience for both customers and administrators.

ER DIAGRAM



This is the Entity Relationship (ER) Diagram of the Readora – MERN Bookstore project, illustrating the relationships between the major entities such as Users, Books, Categories, Shopping Cart, Orders, and Admin. The diagram represents how users can register, browse books, search by category, add books to their shopping cart, and place orders, while administrators can manage the bookstore inventory by adding, updating, and deleting books. The ER diagram provides a clear understanding of the application's database structure and the relationships among different entities. It demonstrates how data is stored and connected within the system using primary and foreign keys, ensuring efficient data retrieval and management. The User, Book, Cart, Order, and Category schemas work together to support the complete online bookstore workflow, enabling secure authentication, seamless order processing, effective inventory management, and reliable database operations through MongoDB Atlas.

PRE-REQUISITES:

The following are the prerequisites required to develop and run the Readora – MERN Bookstore application.

Node.js and npm:

Node.js is a JavaScript runtime environment used to execute server-side JavaScript code. It provides an efficient platform for building scalable web applications.

Npm (Node Package Manager) is used to install and manage project dependencies.

Install Node.js and npm on your development machine.

Download: <https://nodejs.org/en/download/>

Installation instructions: <https://nodejs.org/en/download/package-manager/>

Express.js:

Express.js is a lightweight web application framework for Node.js that simplifies API development, routing, middleware handling, and server-side logic.

Install Express.js by running:

npm install express

MongoDB:

MongoDB Atlas is the cloud-hosted NoSQL database used to store application data such as users, books, orders, categories, and shopping cart information.

Download MongoDB Community Edition (Optional for local database):

Download: <https://www.mongodb.com/try/download/community>

[MongoDB Atlas](https://www.mongodb.com/atlas): <https://www.mongodb.com/atlas>

Installation instructions: <https://docs.mongodb.com/manual/installation/>

React.js:

React.js is a JavaScript library used to build responsive and interactive user interfaces.

The frontend of Readora is developed using React.js with TypeScript.

Installation Guide:

<https://react.dev/learn>

Create React Project:

npm create vite@latest

TypeScript: TypeScript extends JavaScript by adding static typing, making the code more reliable and maintainable.

Installation:

npm install typescript

Documentation:

<https://www.typescriptlang.org/>

Tailwind CSS: Tailwind CSS is a utility-first CSS framework used to build modern and responsive user interfaces.

Installation Guide:

<https://tailwindcss.com/docs/installation>

Redux Toolkit: Redux Toolkit is used for efficient global state management throughout the application.

Installation:

npm install @reduxjs/toolkit react-redux

Documentation:

<https://redux-toolkit.js.org/>

Firebase Authentication: Firebase Authentication provides secure user registration and login functionality.

Documentation:

<https://firebase.google.com/docs/auth>

Installation:

npm install firebase

Axios: Axios is used to communicate between the frontend and backend using REST APIs.

Installation:

npm install axios

Documentation:

<https://axios-http.com/>

Chart.js: Chart.js is used to display dashboard analytics including revenue statistics and book sales.

Installation:

Npm install chart.js react-chartjs-2

Documentation:

<https://www.chartjs.org/>

HTML, CSS & JavaScript: Basic knowledge of HTML, CSS, and JavaScript is required for developing responsive user interfaces and application functionality.

Database Connectivity: The backend uses Mongoose to connect the Node.js server with MongoDB Atlas and perform CRUD operations.

Installation:

npm install mongoose

Documentation:

<https://mongoosejs.com/docs/>

Version Control: Git is used for version control and GitHub is used for repository hosting.

Git Download:

<https://git-scm.com/downloads>

GitHub:

<https://github.com/>

Development Environment

Recommended IDE: Visual Studio Code

Download:

<https://code.visualstudio.com/download>

Running the Existing Readora – MERN Bookstore Project

Clone the Repository

Open Command Prompt or Terminal.

Navigate to the directory where you want to save the project.

Clone the repository:

Git clone <https://github.com/vedhikabasa/Readora-Bookstore.git>

Install Dependencies

Backend

cd backend

npm install

Frontend

cd ../frontend

npm install

Configure Environment Variables

Create a .env file inside both the backend and frontend folders.

Update the required configuration values such as:

- MongoDB Atlas Connection String
- Firebase Configuration
- JWT Secret (if applicable)
- API URLs

Start the Backend Server

```
cd backend  
npm start
```

Or

```
npm run dev
```

Start the Frontend Server

```
cd frontend  
npm run dev
```

Access the Application

Open your browser and visit:

<http://localhost:5173>

You have successfully installed and configured the Readora – MERN Bookstore application on your local machine. The application is now ready for browsing books, secure user authentication, shopping cart management, order processing, and administrative operations. You can further customize, develop, and test the application according to your requirements.

Install Dependencies

Navigate into the cloned repository directory:

```
cd Readora-Bookstore
```

Install the required dependencies by running the following commands:

Backend

```
cd backend  
npm install
```

Frontend

```
cd ../frontend  
npm install
```

Configure Environment Variables

Create a .env file inside the backend folder and update the required values such as:

- MongoDB Atlas Connection String (DB_URL)

- JWT Secret Key (if used)
- Firebase Configuration (if applicable)
- Similarly, create a .env file in the frontend folder and configure the required Firebase credentials.

Start the Development Server**Start the Backend**

```
cd backend
```

```
npm start
```

Or

```
npm run dev
```

Start the Frontend

```
cd ../frontend
```

```
npm run dev
```

Access the Application**Open your browser and visit:**

<http://localhost:5173>

You have successfully installed and configured the Readora – MERN Bookstore application on your local machine. You can now browse books, register and log in securely, manage your shopping cart, place orders, and perform administrative tasks such as managing books and viewing dashboard analytics. The application is ready for further customization, development, and testing.

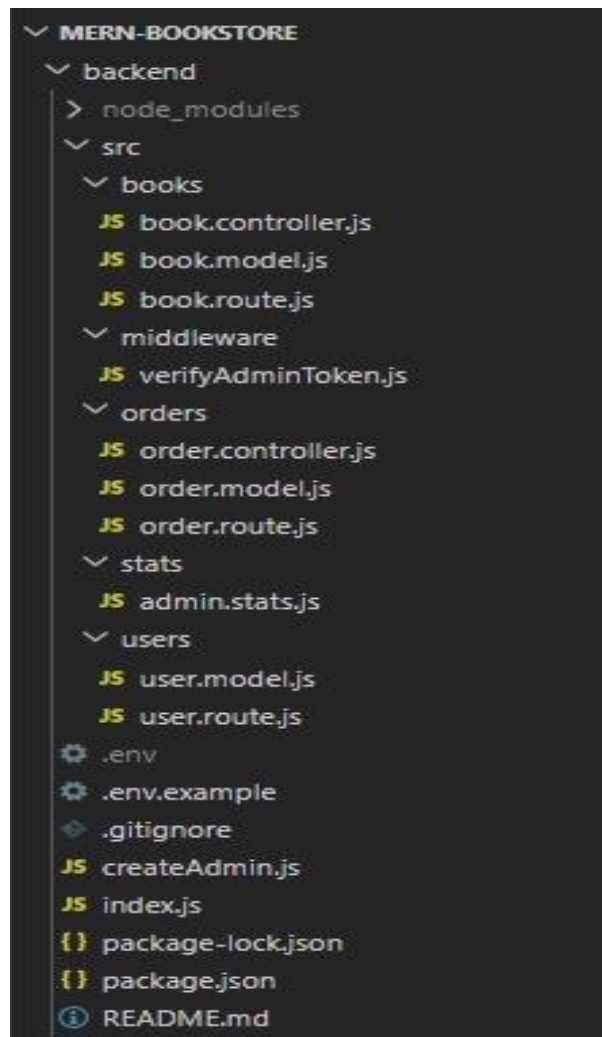
PROJECT STRUCTURE:

```

  frontend
  > node_modules
  > public
  {} books.json
  fav-icon.png
  > src
  > assets
  > books
  > news
  avatar.png
  banner.png
  footer-logo.png
  github-cover.png
  > components
  AdminLogin.tsx
  Footer.tsx
  TS index.ts
  Loading.tsx
  Login.tsx
  Navbar.tsx
  Register.tsx
  > context
  AuthContext.tsx
  > firebase
  TS firebase.config.ts
  > pages
  > books
  BookCard.tsx
  CartPage.tsx
  CheckoutPage.tsx
  TS index.ts
  OrderPage.tsx
  SingleBook.tsx
  > dashboard
  > addBook
  AddBook.tsx
  InputField.tsx
  SelectField.tsx
  > EditBook
  UpdateBook.tsx
  > manageBooks
  ManageBooks.tsx

```

- users
 - UserDashboard.tsx
 - CategoryChart.tsx
 - Dashboard.tsx
 - DashboardLayout.tsx
 - index.ts
 - LatestOrders.tsx
 - RevenueChart.tsx
 - TopSellingBooks.tsx
- home
 - Banner.tsx
 - Home.tsx
 - News.tsx
 - Recommended.tsx
 - TopSellers.tsx
- redux
 - features
 - books
 - booksApi.ts
 - cart
 - cartSlice.ts
 - orders
 - ordersApi.ts
 - stats
 - statsApi.ts
 - store.ts
- routers
 - AdminRoute.tsx
 - AppRouter.tsx
 - PrivateRouter.tsx
- utils
 - baseURL.ts
 - getImgUrl.ts
- App.css
- App.tsx
- index.css
- main.tsx
- vite-env.d.ts
- .env
- .env.example
- .gitignore
- eslint.config.js
- index.html
- package-lock.json
- package.json
- pnpm-lock.yaml



Project Structure

The first image represents the Backend folder structure of the Readora – MERN Bookstore application. It illustrates the files and folders used for backend development, including Models, Controllers, Routes, Middleware, Database Configuration, and REST APIs following the MVC (Model–View–Controller) architecture.

The second and third images represent the Frontend folder structure of the project. They include all the files and folders used to build the user interface, authentication, shopping cart, dashboard, Redux state management, routing, Firebase configuration, and reusable React components.

APPLICATION FLOW

Readora – MERN Bookstore

1. Customer / User

Role: Browse books, manage shopping cart, place orders, and manage personal account.

Flow

Registration & Login

- Create an account using email and password through Firebase Authentication.
- Log in securely using registered credentials.

Browse Books

- View all available books.
- Browse books by category.
- Search books using keywords.
- View detailed information about each book.

Shopping Cart

- Add books to the shopping cart.
- Remove books from the cart.
- Clear the shopping cart.
- Review selected books before checkout.

Order Placement

- Proceed to checkout.
- Confirm the order.
- View previously placed orders.

Profile Management

- Manage personal account information.
- Securely log in and log out.

2. Administrator

Role: Manage bookstore inventory and monitor application performance.

Flow

Admin Login

Log in securely using administrator credentials.

Book Management

- Add new books.
- Edit existing books.
- Delete unavailable books.
- Update book information including title, category, description, cover image, and price.

Dashboard Analytics

- View revenue statistics.
- Monitor latest orders.
- View top-selling books.
- Analyze bookstore performance using charts.

Inventory Management

- Maintain book inventory.
- Ensure accurate book information is available to customers.

Project Flow

Before working on the project, explore the application to understand its functionality and workflow.

The project consists of two major modules:

- Frontend (React + TypeScript)
- Backend (Node.js + Express.js)

These modules communicate through RESTful APIs and store application data in MongoDB Atlas.

Milestone 1

Project Setup & Configuration

1. Create Project Structure: Create separate folders for frontend and backend development.

- Frontend
- Backend

2. Install Required Tools & Software

Backend

- Node.js
- Express.js
- MongoDB Atlas
- Mongoose
- Dotenv
- CORS

Frontend

- React.js
- TypeScript
- Tailwind CSS
- Redux Toolkit
- Axios
- Firebase Authentication
- React Router
- Chart.js
- React Icons
- SweetAlert2

3. Configure Database

- Create a MongoDB Atlas cluster.
- Configure the database connection using environment variables.
- Connect the backend server using Mongoose.

4. Configure Firebase

- Create a Firebase project.
- Enable Authentication.
- Add Firebase configuration keys to the frontend .env file.

5. Run the Application

Backend

Cd backend

Npm install

Npm start

Frontend

Cd frontend

Npm install

Npm run dev

The application will be available at:

<http://localhost:5173>

Milestone 2

Backend Development

- Set Up Project Structure
- Create separate frontend and backend folders for the MERN Bookstore application.
- Initialize the backend using Node.js and install the required dependencies such as Express.js, Mongoose, CORS, dotenv, bcrypt, and JWT.

Configure Express Server

- Create an Express.js server to handle HTTP requests and RESTful API endpoints.
- Configure middleware such as `express.json()`, `CORS`, `dotenv`, and authentication middleware.
- Establish a secure connection with MongoDB Atlas using Mongoose.
- Define REST API Routes.
- Create separate route files for different application modules.

User Authentication Routes

- Book Management Routes
- Order Management Routes
- Admin Dashboard Routes

Each route communicates with its corresponding controller and database model following the MVC architecture.

Implement Data Models

Create Mongoose models for:

- User
- Book
- Order

Implement CRUD operations (Create, Read, Update, Delete) for each model to perform database operations efficiently.

User Authentication

- Implement secure user authentication using JWT and bcrypt.
- Develop APIs for user registration and login.
- Protect private routes using JWT authentication middleware.
- Implement Admin Authentication for bookstore management.

Book Management

- Develop APIs to:
- Add New Books
- Update Existing Books
- Delete Books
- Fetch All Books
- Fetch Single Book Details
- Filter Books by Category
- Search Books

These APIs are accessible according to user authorization.

Order Management

Develop APIs to:

- Add Orders
- Retrieve User Orders
- Retrieve All Orders (Admin)
- Manage Checkout Process

Admin Dashboard

Implement APIs for the administrator to:

- View Revenue Statistics
- View Latest Orders
- Monitor Top Selling Books
- Manage Inventory
- Manage Book Categories
- View Sales Analytics using Chart.js

Error Handling

- Implement centralized error handling middleware.
- Return meaningful HTTP status codes and error messages.
- Handle invalid requests and authentication failures securely.

Backend Configuration Files

backend >  package.json >...

Abdullah Koyuncu, 4 weeks ago | 2 authors (Wise and one other)

```
1  {
2    "name": "mern-bookstore-backend",
3    "version": "1.0.0",
4    "description": "",
5    "main": "index.js",
6    "scripts": {
7      "start": "node index.js",
8      "dev": "nodemon index.js"
9    },
10   "keywords": [],
11   "author": "Abdullah Koyuncu",
12   "license": "ISC",
13   "type": "commonjs",
14   "dependencies": {
15     "bcrypt": "^6.0.0",
16     "cors": "^2.8.6",
17     "dotenv": "^17.4.2",
18     "express": "^5.2.1",
19     "jsonwebtoken": "^9.0.3",
20     "mongoose": "^9.7.0"
21   },
22   "devDependencies": {
23     "nodemon": "^3.1.14"
24   }
25 }
26
```

```

backend > # index.js >--
Wise, 15 months ago | 1 author (Wise)
1 'use strict'
2
3 const express = require("express");
4 const app = express();
5 const cors = require("cors");
6 |       Wise, 15 months ago * Backend structure with initial files and config.
7 const mongoose = require("mongoose");
8 const port = process.env.PORT || 5000;
9 require('dotenv').config()
10
11 app.use(express.json());
12 app.use(cors({
13   |   origin: ['http://localhost:5173', '[:,:1]:5173'], // You can (and you need to) add your vercel/heroku URL here
14   |   credentials: true
15 })))
16
17 const bookRoutes = require('./src/books/book.route');
18 const orderRoutes = require("./src/orders/order.route")
19 const userRoutes = require("./src/users/user.route")
20 const adminRoutes = require("./src/stats/admin.stats")
21
22 app.use("/api/books", bookRoutes)
23 app.use("/api/orders", orderRoutes)
24 app.use("/api/auth", userRoutes)
25 app.use("/api/admin", adminRoutes)
26
27 async function main() {
28   |   await mongoose.connect(process.env.DB_URL);
29   |   app.use("/", (req, res) => {
30   |     |   res.send("Book Store Server is running!");
31   |   });
32 }
33
34 main().then(() => console.log("Mongodb connect successfully!")).catch(err => console.log(err));
35
36 app.listen(port, () => {
37   |   console.log(`Example app listening on port ${port}`);
38 });
39

```

MongoDB Atlas Connection

Backend Output

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  GITLENS

PS C:\Users\vedhi\OneDrive\Desktop\mern-bookstore> cd backend
PS C:\Users\vedhi\OneDrive\Desktop\mern-bookstore\backend> npm run dev

> mern-bookstore-backend@1.0.0 dev
> nodemon index.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node index.js`
◇ injected env (3) from .env // tip: ⌘ override existing { override: true }
Example app listening on port 5000
MongoDB connect successfully!
█
```

Reference Video link for Backend:

<https://drive.google.com/file/d/1bEz9DQyKyZ2pnxpVTUvYzYamWGRnXYDy/view?usp=sharing>

Milestone 3:

Database Development

1. User Schema

- The User schema defines the structure of user information stored in the MongoDB database.
- It includes fields such as username, email, password, role, and profile information.
- Passwords are securely encrypted using bcrypt before being stored.
- User data is stored in the users collection.

2. Book Schema

- The Book schema defines the structure of book information available in the bookstore.
- It contains fields such as title, author, category, description, coverImage, oldPrice, newPrice, trending, and createdAt.
- Each book record contains complete details required for displaying books in the application.
- Book data is stored in the books collection.

3. Order Schema

- The Order schema stores customer purchase information.
- It includes fields such as userId, products, totalPrice, paymentStatus, shippingAddress, orderStatus, and orderDate.
- Each order is linked to a registered user using the userId field.
- Order data is stored in the orders collection.

4. Admin Statistics

The Admin Statistics module generates analytical information for the dashboard.

It retrieves data from the Books and Orders collections to display:

- Total Revenue
- Total Orders
- Top Selling Books
- Category-wise Sales
- Latest Orders

These statistics help administrators monitor bookstore performance efficiently.

Milestone 4:

Frontend Development

1. Setup React Application

Readora Bookstore is developed using React.js, TypeScript, Vite, and Tailwind CSS for building a modern, responsive, and interactive user interface.

To begin frontend development:

- Install all required dependencies.
- Create the frontend project structure.
- Configure React Router, Redux Toolkit, and Firebase.
- Organize reusable components, pages, assets, and utility files.

2. Design UI Components

Develop reusable UI components for different sections of the application.

These include:

- Navigation Bar
- Banner Section
- Book Cards
- Category Section
- Recommended Books
- Top Selling Books
- Shopping Cart
- Checkout Page
- Login & Registration Forms
- User Dashboard
- Admin Dashboard

- Revenue Charts
- Category Charts
- Latest Orders
- Footer

The interface is designed using Tailwind CSS to provide a responsive and user-friendly experience across all devices.

3. Implement Frontend Logic

- Connect the frontend with backend REST APIs.
- Implementation includes:
 - Integration with Express.js API endpoints using Axios.
 - User Authentication using JWT.
 - Firebase Authentication integration.
 - Redux Toolkit for Cart and State Management.
 - Dynamic rendering of books from MongoDB.
 - Book Search and Category Filtering.
 - Add to Cart functionality.
 - Checkout and Order Placement.
 - Admin Dashboard with analytics and inventory management.
 - Real-time UI updates after CRUD operations.

Reference video for frontend code:

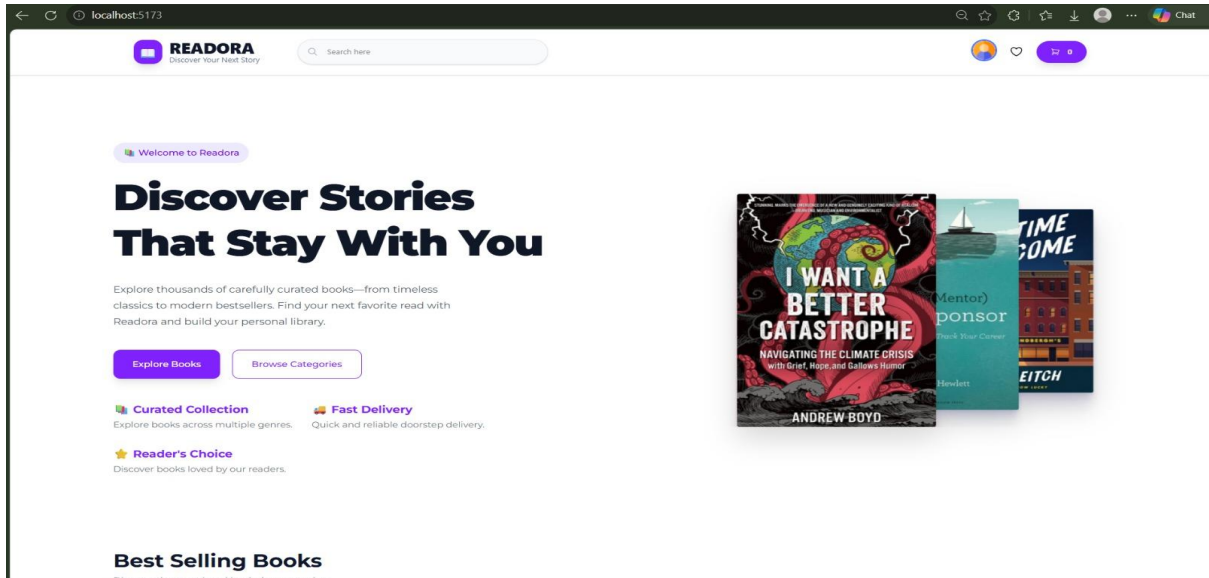
https://drive.google.com/file/d/1kBTrFmxclv_QPlcmzvb4T5orlZ6ve9vF/view?usp=sharing

Milestone 5:

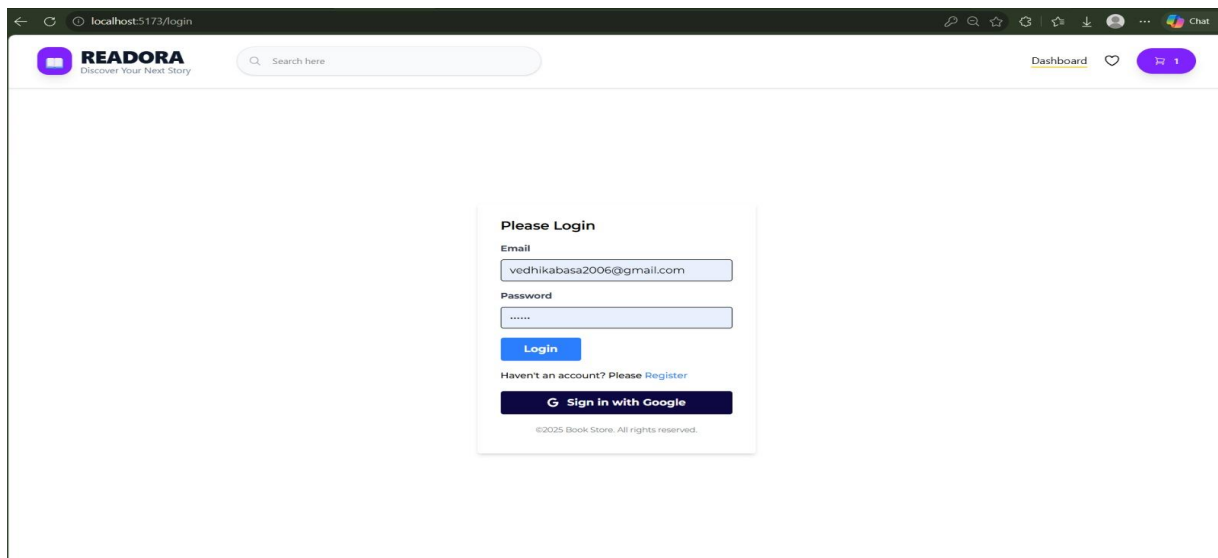
Project Implementation:

On completing the development part, we then ran the application one last time to verify all the functionalities and look for any bugs in it. The user interface of the application looks a bit like the one provided below.

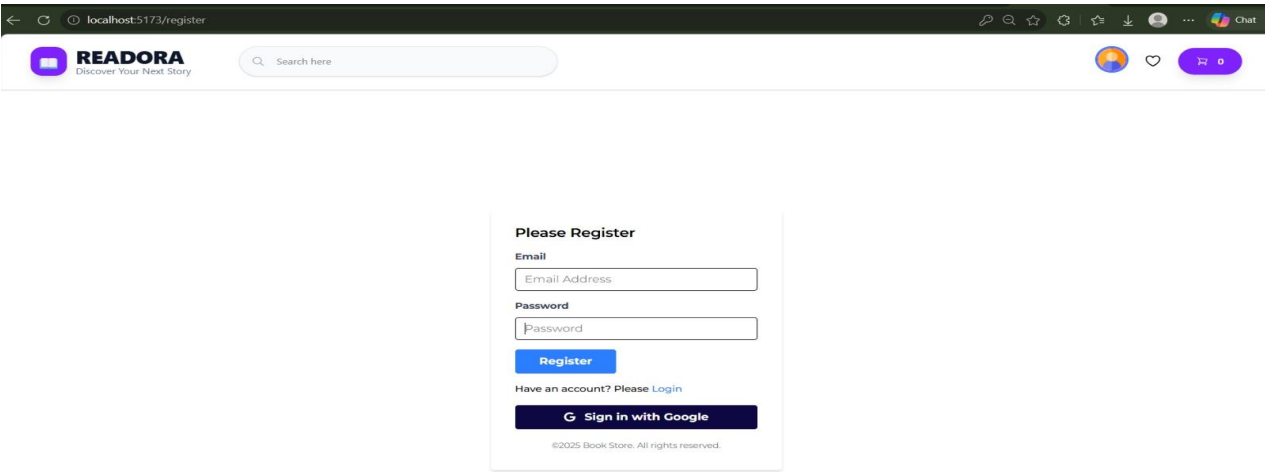
- Landing Page



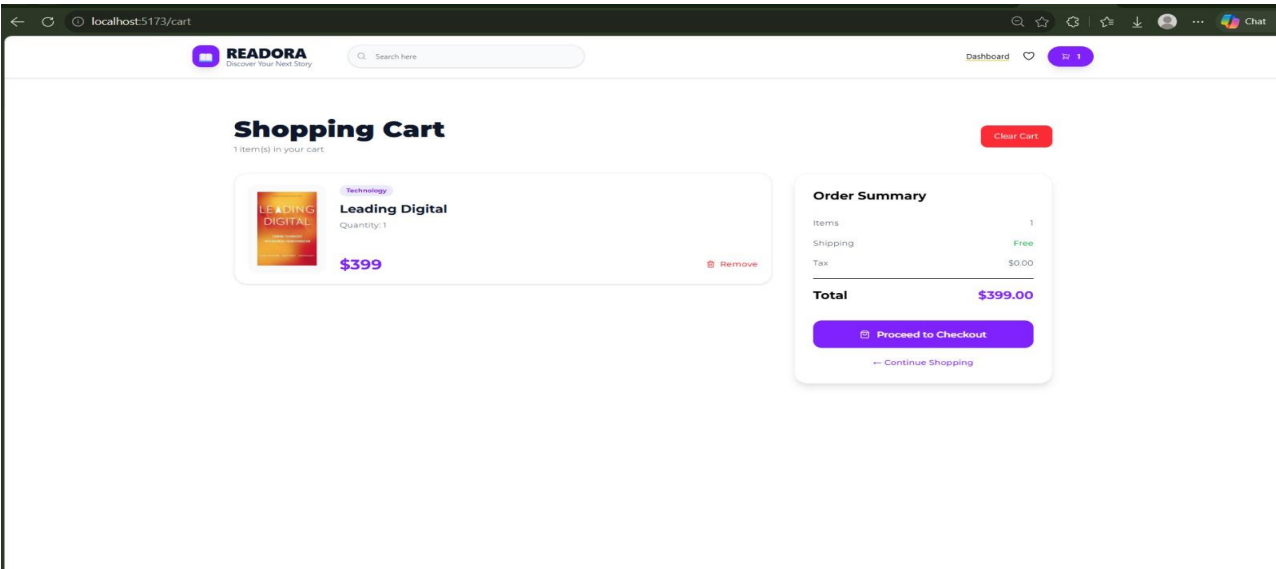
- Login Page



- Registration Page



- Cart Page



- Checkout Page

←localhost:5173/checkout

READORA
Discover Your Next Story

Search here



Checkout

Complete your purchase by filling in your shipping details.

Shipping Information

Full Name

Email

vedhikabasa2006@gmail.com

Phone

City

Address

Country

State

Zip Code

Order Summary

Items	1
Shipping	Free
Payment	Cash On Delivery
Total	\$399

Free Shipping

Secure Checkout

Cash On Delivery

I agree to the Terms & Conditions and Privacy Policy.

Place Order

Back to Cart

- Your Orders Page

←localhost:5173/orders

READORA
Discover Your Next Story

Search here

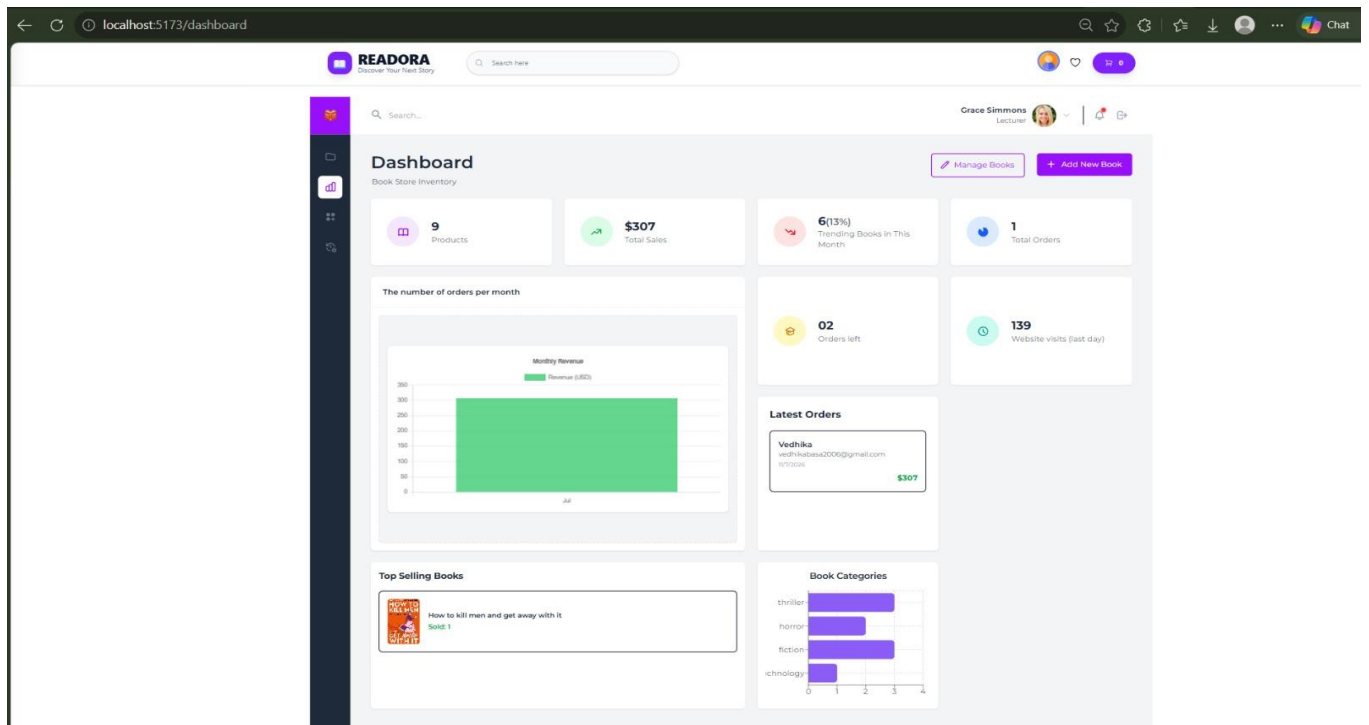


Your Orders

#1

Order ID: 6a515db93fce4e76d8f5697b
Name: Vedhika
Email: vedhikabasa2006@gmail.com
Phone: 918186959659
Total Price: \$307
Address:
10-65, Eshwar nagar 4th cross, APSP Kakinada, 533003, Kakinada, Andhra Pradesh, India, 533003
Products Id:
6a513e16fa38e9886778a405

Admin Dashboard Page



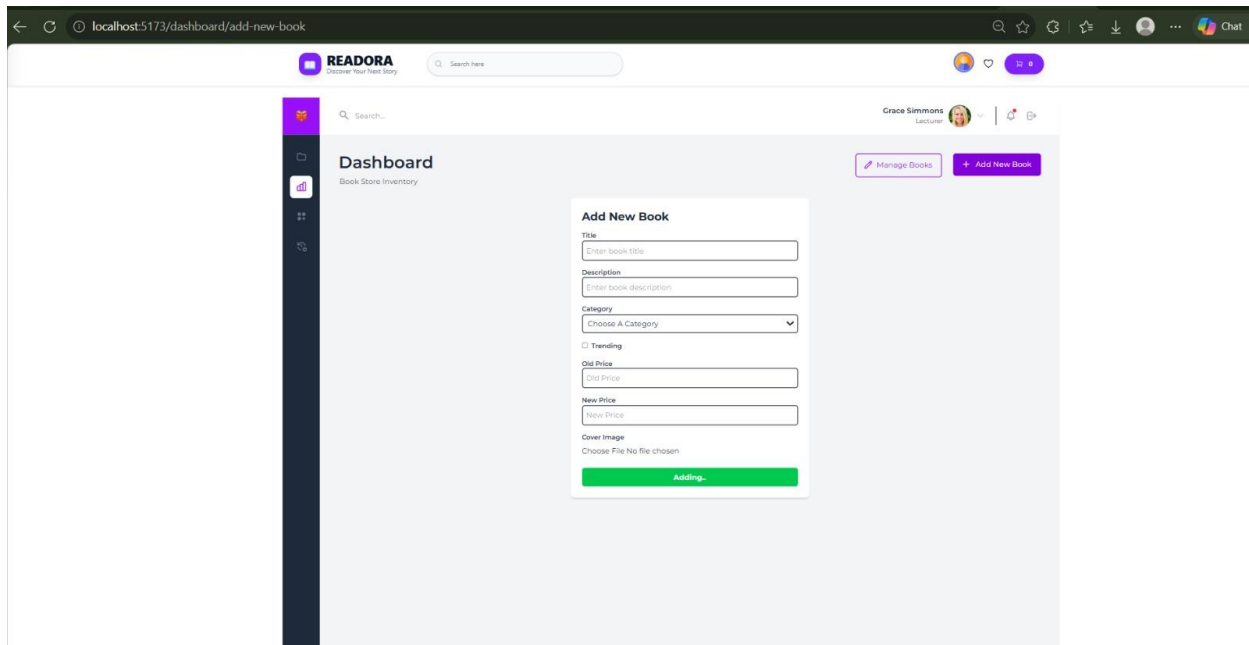
Admin Manage Books Page

The screenshot shows the READORA Admin Manage Books page with the following components:

- Header:** READORA logo, search bar, and user profile (Grace Simmons, Lecturer).
- Dashboard Title:** Dashboard, Book Store Inventory.
- Buttons:** Manage Books, Add New Book.
- Table:** A table listing all books with columns for ID, Book Title, Category, Price, and Actions.

ID	BOOK TITLE	CATEGORY	PRICE	ACTIONS
1	Leading Digital	technology	\$399	Edit Delete
2	The Imaginary Friend	horror	\$430	Edit Delete
3	Until Death	horror	\$335	Edit Delete
4	Invisible Man	fiction	\$399	Edit Delete
5	Blue Sisters	fiction	\$309	Edit Delete
6	The Housemaid	thriller	\$270	Edit Delete
7	The Love Hypothesis: Collector's edition	fiction	\$1999	Edit Delete
8	How to kill men and get away with it	thriller	\$307	Edit Delete
9	Verity	thriller	\$2495	Edit Delete

Add New Book Page



The screenshot displays a web browser window at the URL `localhost:5173/dashboard/add-new-book`. The page features a dark sidebar on the left with a purple header containing the 'READORA' logo and a search bar. The main content area is titled 'Dashboard' with the subtitle 'Book Store Inventory'. It includes a search bar, a user profile for 'Grace Simmons' (Lecturer), and two buttons: 'Manage Books' and '+ Add New Book'. The 'Add New Book' form is centered and contains the following fields: 'Title' (with placeholder 'Enter book title'), 'Description' (with placeholder 'Enter book description'), 'Category' (a dropdown menu with 'Choose A Category'), a 'Trending' checkbox, 'Old Price' (with placeholder 'Old Price'), 'New Price' (with placeholder 'New Price'), and 'Cover Image' (with placeholder 'Choose File No file chosen'). A green 'Adding...' button is at the bottom of the form.

The demo of the app is available at:

https://drive.google.com/file/d/1wb-ZXFszmtOOTob6HqVZgZ_a7pk2qlhL/view?usp=sharing